

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

ОТЧЕТ

по лабораторной работе №1

Тема: «Цветовые модели»

Выполнил:
Студент группы 2
Рыжов П. А.

Вариант:
Нечетный (CMYK – RGB – HLS)

Минск, 2026

Содержание

1	Цель работы	2
2	Задание на лабораторную работу	2
3	Теоретическая база и математические формулы	2
3.1	Модель RGB (Red, Green, Blue)	2
3.2	Модель CMYK (Cyan, Magenta, Yellow, Key color)	2
3.3	Модель HLS (Hue, Lightness, Saturation)	3
4	Особенности реализации	3
5	Исходный код основных алгоритмов	4
6	Интерфейс программы	6
7	Выводы	7

1 Цель работы

Изучить принципы формирования цвета в различных цветовых моделях (RGB, CMYK, HLS) и математические алгоритмы преобразования между ними. Разработать интерактивное десктопное приложение с графическим интерфейсом пользователя (GUI), предоставляющее инструментарий для выбора, редактирования и автоматического синхронного пересчета цветовых координат в реальном времени.

2 Задание на лабораторную работу

1. Реализовать математический аппарат для конвертации цветов между цветовыми моделями CMYK, RGB и HLS.
2. Разработать дружелюбный графический интерфейс, предоставляющий возможность задавать цвет тремя различными способами:
 - Точный ввод значений через текстовые поля (с защитой от некорректного ввода).
 - Плавное изменение значений с помощью интерактивных ползунков (слайдеров).
 - Визуальный выбор цвета с помощью палитры, аналогичной тем, что применяются в профессиональных графических редакторах.
3. Обеспечить автоматический и синхронный пересчет всех представлений цвета: при изменении любой отдельной координаты (компоненты) в одной из моделей, значения в двух других должны обновляться мгновенно.

3 Теоретическая база и математические формулы

3.1 Модель RGB (Red, Green, Blue)

Аппаратно-ориентированная, аддитивная цветовая модель. Используется для воспроизведения цвета на экранах мониторов. Базовыми компонентами являются красный (R), зеленый (G) и синий (B) цвета. Значения координат задаются целыми числами в диапазоне от 0 до 255.

3.2 Модель CMYK (Cyan, Magenta, Yellow, Key color)

Субтрактивная модель, применяемая в полиграфии для полноцветной печати. Формирование цвета происходит за счет вычитания части спектра из падающего света (на белой бумаге). В теории достаточно трех цветов (CMY), однако для получения глубокого черного цвета и экономии цветных красок введен четвертый канал — Key color (черный). Значения задаются в процентах от 0 до 100.

Преобразование RGB → CMYK:

Сначала координаты RGB нормируются к отрезку $[0, 1]$: $R' = R/255$, $G' = G/255$, $B' = B/255$. Вычисляется доля черного цвета (K):

$$K = 1 - \max(R', G', B') \quad (1)$$

Если $K = 1$ (чистый черный цвет), то $C = M = Y = 0$. Иначе координаты вычисляются по формулам:

$$C = \frac{1 - R' - K}{1 - K}, \quad M = \frac{1 - G' - K}{1 - K}, \quad Y = \frac{1 - B' - K}{1 - K} \quad (2)$$

Для получения процентов полученные значения умножаются на 100.

Преобразование CMYK $\rightarrow$ RGB:

$$R = 255 \times (1 - C) \times (1 - K) \quad (3)$$

$$G = 255 \times (1 - M) \times (1 - K) \quad (4)$$

$$B = 255 \times (1 - Y) \times (1 - K) \quad (5)$$

3.3 Модель HLS (Hue, Lightness, Saturation)

Перцепционная (интуитивная) цветовая модель, описывающая цвет в терминах, понятных человеческому восприятию: Тон (H), Светлота (L) и Насыщенность (S).

- $H \in [0, 360)$ — цветовой тон (угол на цветовом круге).
- $L \in [0, 100]$ — светлота (0% — черный, 100% — белый).
- $S \in [0, 100]$ — насыщенность (0% — оттенок серого, 100% — максимально чистый цвет).

Преобразование RGB $\rightarrow$ HLS:

Пусть $M = \max(R', G', B')$, $m = \min(R', G', B')$, а $\Delta = M - m$. Светлота вычисляется как среднее между максимумом и минимумом:

$$L = \frac{M + m}{2} \quad (6)$$

Если $\Delta = 0$ (ахроматический цвет), то $H = 0$ и $S = 0$. Иначе насыщенность вычисляется по формуле:

$$S = \begin{cases} \frac{\Delta}{M+m}, & \text{если } L \leq 0.5 \\ \frac{\Delta}{2.0-M-m}, & \text{если } L > 0.5 \end{cases} \quad (7)$$

Цветовой тон H зависит от того, какой канал является максимальным:

$$H = 60^\circ \times \begin{cases} \frac{G'-B'}{\Delta} \pmod{6}, & \text{если } M = R' \\ \frac{B'-R'}{\Delta} + 2, & \text{если } M = G' \\ \frac{R'-G'}{\Delta} + 4, & \text{если } M = B' \end{cases} \quad (8)$$

4 Особенности реализации

Приложение разработано на языке программирования **Go (Golang)** с использованием кроссплатформенного фреймворка **Fyne**.

Ключевые архитектурные и технические решения:

- **Безопасность типов и Clamping:** Использована система обобщений (Generics) языка Go для создания универсальной функции `clampUint`, которая округляет вещественные числа и строго удерживает их в границах типов `uint8` и `uint16`, предотвращая паники при переполнении.
- **Предотвращение рекурсии (Зацикливания событий):** Из-за автоматического пересчета изменение одного слайдера вызывает программное изменение других, что могло бы привести к бесконечному циклу. Проблема решена с помощью флажка `isUpdating` и отложенных вызовов `defer`.

- **Кастомная 2D/1D палитра:** Разработан собственный сложный виджет палитры. Он состоит из интерактивной 2D-области (управление Saturation и Lightness по координатам X и Y) и 1D-шкалы (управление Hue). Виджет перехватывает события мыши (Drag/Tap) и производит мгновенный рендеринг пикселей с использованием формул конвертации.

5 Исходный код основных алгоритмов

В Листинге 1 представлена вспомогательная дженерик-функция. Она критически важна для нивелирования погрешностей арифметики с плавающей точкой (float64) при обратном переводе в целые числа.

```

1 // Generic function to prevent overflow and underflow
2 func clampUint[T uint | uint8 | uint16 | uint32 | uint64](val float64, min,
   max T) T {
3     rounded := math.Round(val)
4     if rounded < float64(min) {
5         return min
6     }
7     if rounded > float64(max) {
8         return max
9     }
10    return T(rounded)
11 }

```

Листинг 1: Функция безопасного приведения типов (Clamping)

В Листинге 2 приведена реализация перевода RGB в HLS, строго следующая математическим формулам (см. раздел 3.3). Обратите внимание на обработку краевого случая `delta == 0` для оттенков серого.

```

1 func (rgb RGB) ToHLS() HLS {
2     r := float64(rgb.R) / 255.0
3     g := float64(rgb.G) / 255.0
4     b := float64(rgb.B) / 255.0
5
6     mx := max(r, g, b)
7     mn := min(r, g, b)
8     delta := mx - mn
9
10    var h, l, s float64
11    l = (mx + mn) / 2.0
12
13    if delta == 0 {
14        // Achromatic color (gray, black, white)
15        h = 0
16        s = 0
17    } else {
18        if l <= 0.5 {
19            s = delta / (mx + mn)
20        } else {
21            s = delta / (2.0 - mx - mn)
22        }
23
24        switch mx {
25        case r:
26            h = (g - b) / delta
27            if g < b {
28                h += 6.0

```

```

29         }
30         case g:
31             h = ((b - r) / delta) + 2.0
32         case b:
33             h = ((r - g) / delta) + 4.0
34     }
35     h *= 60.0
36 }
37
38 roundedH := uint16(math.Round(h))
39 if roundedH >= 360 {
40     roundedH = 0
41 }
42
43 return HLS{
44     H: roundedH,
45     L: clampUint[uint8](l*100.0, 0, 100),
46     S: clampUint[uint8](s*100.0, 0, 100),
47 }
48 }

```

Листинг 2: Алгоритм перевода RGB в HLS

В Листинге 3 представлена обратная функция. Алгоритм вычисляет цветность (chroma, c), вторичный компонент (x) и смещение светлоты (m), после чего распределяет их по каналам RGB в зависимости от сектора, в который попадает угол Hue.

```

1 func (hls HLS) ToRGB() RGB {
2     h := float64(hls.H)
3     l := float64(hls.L) / 100.0
4     s := float64(hls.S) / 100.0
5
6     // Grayscale fallback
7     if s == 0 {
8         val := clampUint[uint8](l*255.0, 0, 255)
9         return RGB{R: val, G: val, B: val}
10    }
11
12    c := (1.0 - math.Abs(2.0*l-1.0)) * s
13    x := c * (1.0 - math.Abs(math.Mod(h/60.0, 2.0)-1.0))
14    m := l - c/2.0
15
16    var r1, g1, b1 float64
17    switch {
18    case h >= 0 && h < 60:
19        r1, g1, b1 = c, x, 0
20    case h >= 60 && h < 120:
21        r1, g1, b1 = x, c, 0
22    case h >= 120 && h < 180:
23        r1, g1, b1 = 0, c, x
24    case h >= 180 && h < 240:
25        r1, g1, b1 = 0, x, c
26    case h >= 240 && h < 300:
27        r1, g1, b1 = x, 0, c
28    default:
29        r1, g1, b1 = c, 0, x
30    }
31
32    return RGB{
33        R: clampUint[uint8]((r1+m)*255.0, 0, 255),
34        G: clampUint[uint8]((g1+m)*255.0, 0, 255),
35        B: clampUint[uint8]((b1+m)*255.0, 0, 255),

```

36
37

```
}  
}
```

Листинг 3: Алгоритм перевода HLS в RGB

6 Интерфейс программы

Окно приложения разделено на две основные логические части. В левой части расположена профессиональная палитра для визуального выбора цвета, прямоугольник-превью текущего цвета и поле для ввода HEX-кода. В правой части скомпонованы карточки для каждой из трех цветовых моделей (RGB, CMYK, HLS), содержащие текстовые поля ввода и слайдеры.

Интерфейс работает полностью синхронно. При перетаскивании маркеров на палитре ползунки и текстовые поля справа обновляются в реальном времени. Аналогично, при ручном вводе значений в правые панели, маркеры на палитре смещаются в соответствующие координаты.

На Рисунке 1 представлен внешний вид приложения при выборе хроматического цвета высокой насыщенности.

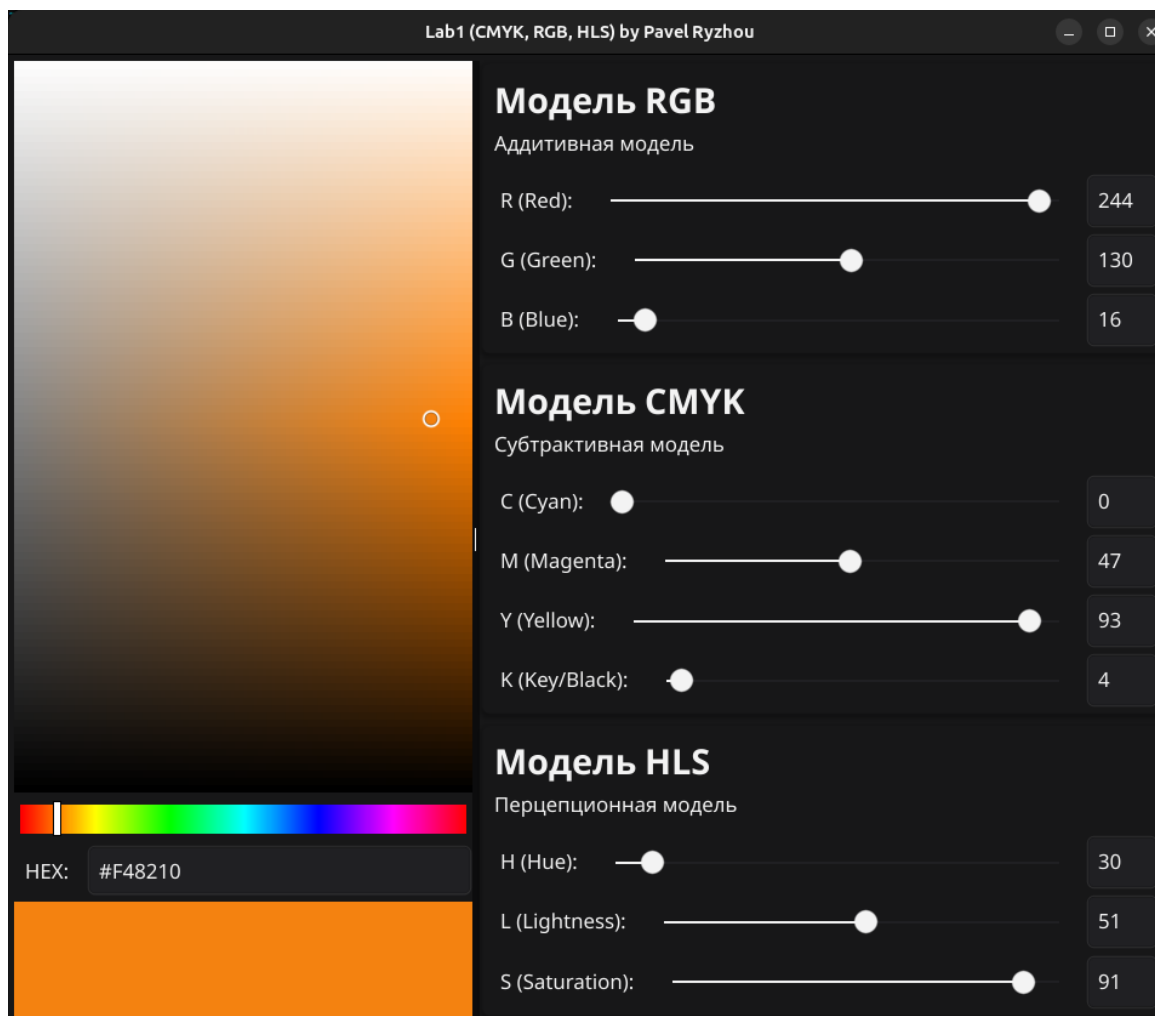


Рис. 1: Интерфейс приложения (выбран хроматический цвет)

На Рисунке 2 продемонстрирована работа программы при выборе ахроматического оттенка. Можно заметить, что насыщенность (S) и компоненты CMY стремятся к нулю,

в то время как канал К (Key) управляет затемнением.

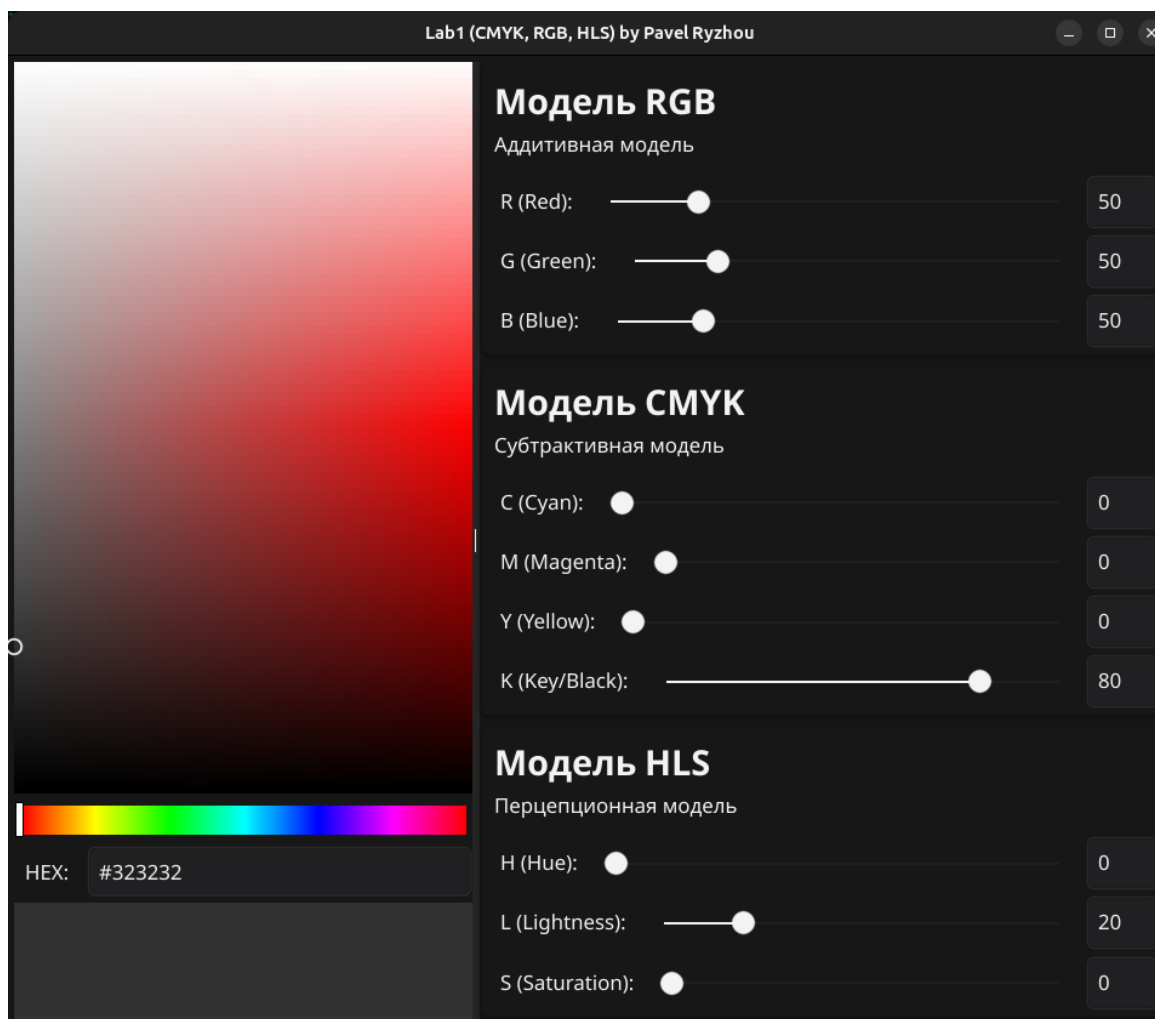


Рис. 2: Интерфейс приложения (ахроматический цвет)

7 Выводы

В ходе выполнения лабораторной работы были глубоко изучены принципы формирования цвета в аддитивных (RGB), субтрактивных (CMYK) и перцепционных (HLS) цветовых моделях.

Успешно реализованы сложные математические алгоритмы их взаимного преобразования. Особое внимание было уделено обработке краевых случаев (например, деление на ноль при вычислении параметров чистого черного или серого цветов) и борьбе с погрешностями вещественной арифметики путем использования безопасного приведения типов.

Разработанное десктопное приложение полностью удовлетворяет требованиям задания. Оно обладает отзывчивым графическим интерфейсом, поддерживает три независимых способа задания цвета (включая профессиональную интерактивную палитру и HEX) и гарантирует строгую синхронизацию данных во всех цветовых пространствах без закликивания событий.